

Révision

Exercice 1:

- 1) Écrire une fonction itérative et récursive qui compte et renvoie le nombre d'occurrences de l'élément n dans le tableau T et développer le programme principal qui appelle ces fonctions.
- 2) Écrire une fonction récursive avec pointeurs et développer le programme principal qui appelle cette fonction

Fonction Compter_Iteratif(T : tableau d'entiers, n : entier, taille : entier) : entier

Variables

i,compteur: entier

compteur $\leftarrow$ 0

Pour i $\leftarrow$ 1 à taille **Faire**

Si (T[i] = n) **Alors**

 compteur $\leftarrow$ compteur + 1

FinSi

FinPour

Retourner compteur

FinFonction

Fonction Compter_Recuratif(T : tableau d'entiers, n : entier, taille : entier) : entier

Si taille = 0 **Alors**

Retourner 0

Sinon

Si (T[taille] = n) **Alors**

Retourner 1 + Compter_Recuratif(T, n, taille - 1)

Sinon

Retourner Compter_Recuratif(T, n, taille - 1)

FinSi

FinSi

FinFonction

Fonction nbOccRec (T : tableau d'entiers, n : entier, i : entier, taille : entier) : entier

Début

// Condition d'arrêt : fin du tableau

Si (i > taille) Alors

Retourner 0

FinSi

Si (T[i] = n) Alors

Retourner 1 + nbOccRec(T, n, i+1, taille)

Sinon

Retourner nbOccRec(T, n, i+1, taille)

FinSi

Fin

Algorithme Principal

Variables

T : tableau d'entiers

taille, n, res_iter, res_rec : entier

Début

Écrire("Donner la taille du tableau: ")

Lire(taille)

Pour i de 1 à taille faire

Écrire("Donner un nombre : ")

Lire(T[i])

FinPour

Écrire("Donner l'élément à rechercher: ")

Lire(n)

res_iter ← Compter_Iteratif(T, n, taille)

res_rec ← Compter_Recursif(T, n, taille)

Ecrire("Nombre d'occurrences (itératif) :", res_iter)

Ecrire("Nombre d'occurrences (récuratif) :", res_rec)

FinAlgorithme

```
Fonction nbOccRec (T : pointeur sur entier, n : entier,  
i : entier, taille : entier) : entier
```

```
Début
```

```
    // Condition d'arrêt : fin du tableau
```

```
    Si i > taille Alors
```

```
        Retourner 0
```

```
    FinSi
```

```
    Si (T + (i - 1))^ = n Alors
```

```
        Retourner 1 + nbOccRec(T, n, i + 1, taille)
```

```
    Sinon
```

```
        Retourner nbOccRec(T, n, i + 1, taille)
```

```
    FinSi
```

```
Fin
```

```
Programme Principal
```

```
Variables :
```

```
    T : pointeur sur entier
```

```
    N, x, resultat, i : entier
```

```
Début
```

```
    Écrire("Donner la taille du tableau : ")
```

```
    Lire(N)
```

```
    // allocation du tableau
```

```
    T ← allouer(N * taille(entier))
```

```
    Pour i de 1 à N Faire
```

```
        Lire( (T + i - 1)^ )
```

```
    FinPour
```

```
    Écrire("Donner l'élément à rechercher : ")
```

```
    Lire(x)
```

```
    // Appel de la fonction récursive
```

```
    resultat ← nbOccRec(T, x, 1, N)
```

```
    Écrire("Occurrences (récursif) = ", resultat)
```

```
    // Libération de la mémoire
```

```
    Libérer(T)
```

```
Fin
```

Exercice 2

On représente une image en niveaux de gris par une matrice M de dimensions $L \times C$, contenant des entiers compris entre 0 et 255.

On souhaite écrire un algorithme contenant les quatre sous-programmes suivants :

- Un sous programme qui demande à l'utilisateur de saisir une matrice M d'une image niveau de gris à deux dimensions $L \times C$ (M contient des entiers qui représentent les pixels).
- Un sous programme qui calcule la valeur moyenne de cette matrice (la moyenne des pixels dans l'image niveau de gris).

$$\text{Moy} = \frac{1}{CL} \sum_{i=1}^L \sum_{j=1}^C M[i, j]$$

- Un sous programme qui crée un mot de passe pour accéder à cette image. Pour cette image, ce mot de passe est un tableau T d'entiers de dimension C , ses valeurs sont calculées à partir de l'image selon la formule suivante :

$$T[j] = \frac{\sum_{i=1}^L M[i, j] * i}{\text{moy}}$$

$$M = \begin{pmatrix} 10 & 20 & 30 & 40 \\ 15 & 25 & 35 & 45 \\ 20 & 30 & 40 & 50 \end{pmatrix} \quad \text{donne} \quad T = [3, 5, 7, 9]$$

- Un sous programme qui applique une conversion de l'image d'origine M vers une image avec 3 niveaux de valeurs en effectuant les remplacements suivants :
 - Les valeurs dans (0, ..., 63) sont remplacées par la valeur 0,
 - Les valeurs dans (64, ..., 127) sont remplacées par la valeur 1,
 - Les valeurs dans (128, ..., 191) sont remplacées par la valeur 2,
 - Les valeurs dans (192 ..., 255) sont remplacées par la valeur 3

On représente une image en niveaux de gris par une matrice M de dimensions $L \times C$, contenant des entiers compris entre 0 et 255.

On souhaite écrire un algorithme contenant les quatre sous-programmes suivants :

- Un sous programme qui demande à l'utilisateur de saisir une matrice M d'une image niveau de gris à deux dimensions $L \times C$ (M contient des entiers qui représentent les pixels).

Procédure Saisir_Image(Var M : tableau[L][C] d'entiers, L : entier, C : entier)

Variables

i, j : entier

Pour $i \leftarrow 1$ à L Faire

Pour $j \leftarrow 1$ à C Faire

Répéter

Lire $M[i][j]$

Jusqu'à ($0 \leq M[i][j] \leq 255$)

FinPour

FinPour

FinProcédure

Procédure saisirMatrice (var M : pointeur sur entier, L, C : entier)

Variables :

i, j : entier

Début

M ← allouer(L * C * taille(entier))

Pour i de 1 à L Faire

Pour j de 1 à C Faire

Répéter

Lire((M+i-1) ^+j-1) ^)

Jusqu'à(((M+i-1) ^+j-1) ^ >= 0

ET ((M+i-1) ^+j-1) ^ <= 255)

FinPour

FinPour

Fin

- Un sous programme qui calcule la valeur moyenne de cette matrice (la moyenne des pixels dans l'image niveau de gris).

$$\text{Moy} = \frac{1}{CL} \sum_{i=1}^L \sum_{j=1}^C M[i, j]$$

Fonction Moyenne_Image(M : tableau[L][C] d'entiers, L : entier, C : entier) : réel

Variables

i, j, somme : entier

somme $\leftarrow$ 0

Pour i $\leftarrow$ 1 à L **Faire**

Pour j $\leftarrow$ 1 à C **Faire**

 somme $\leftarrow$ somme + M[i][j]

FinPour

FinPour

Retourner somme / (L $\times$ C)

FinFonction

```
Fonction moyenne (M : pointeur sur entier, L : entier, C : entier) : réel
```

```
Variables :
```

```
    i, j      : entier  
    somme : entier
```

```
Début
```

```
    somme ← 0
```

```
    Pour i de 1 à L Faire
```

```
        Pour j de 1 à C Faire
```

```
            somme ← somme + (M+i-1)^(j-1)
```

```
        FinPour
```

```
    FinPour
```

```
    Retourner somme / (L * C)
```

```
Fin
```

- Un sous programme qui crée un mot de passe pour accéder à cette image. Pour cette image, ce mot de passe est un tableau T d'entiers de dimension C, ses valeurs sont calculées à partir de l'image selon la formule suivante :

$$T[j] = \frac{\sum_{i=1}^L M[i, j] * i}{moy}$$

$$M = \begin{pmatrix} 10 & 20 & 30 & 40 \\ 15 & 25 & 35 & 45 \\ 20 & 30 & 40 & 50 \end{pmatrix} \quad \text{donne} \quad T = [3, 5, 7, 9]$$

$$moy = \frac{360}{12} = 30$$

Formule :

$$T[j] = \frac{\sum_{i=1}^L M[i, j] \times i}{moy}$$

$$M = \begin{pmatrix} 10 & 20 & 30 & 40 \\ 15 & 25 & 35 & 45 \\ 20 & 30 & 40 & 50 \end{pmatrix}$$

Colonne 1

$$10 \times 1 + 15 \times 2 + 20 \times 3 = 100$$

$$T[1] = \frac{100}{30} = \frac{10}{3} \approx 3,33$$

Colonne 2

$$20 \times 1 + 25 \times 2 + 30 \times 3 = 160$$

$$T[2] = \frac{160}{30} = \frac{16}{3} \approx 5,33$$

Colonne 3

$$30 \times 1 + 35 \times 2 + 40 \times 3 = 220$$

$$T[3] = \frac{220}{30} = \frac{22}{3} \approx 7,33$$

Colonne 4

$$40 \times 1 + 45 \times 2 + 50 \times 3 = 280$$

$$T[4] = \frac{280}{30} = \frac{28}{3} \approx 9,33$$

$$T = (3,33, 5,33, 7,33, 9,33)$$

► Après conversion en entiers :

$$T = (3, 5, 7, 9)$$

**Procédure Generer_Mot_Passe(M : tableau[L][C] d'entiers, L,C : entier, moy : réel,
var T : tableau[C] d'entiers)**

Variables

i, j,somme : entier

Pour j $\leftarrow$ 1 à C Faire

somme $\leftarrow$ 0

Pour i $\leftarrow$ 1 à L Faire

somme $\leftarrow$ somme + M[i][j] $\times$ i

FinPour

T[j] $\leftarrow$ somme / moy

FinPour

FinProcédure

Avec pointeurs

```
Procédure creerMotDePasse (M : pointeur sur entier, L : entier, C : entier, var T : pointeur sur entier,  
Variables :  
i, j      : entier  
sommeCol  : entier  
moy       : réel  
Début  
    // Allocation du mot de passe T de taille C  
    T ← allouer( C * taille(entier) )  
  
    moy ← moyenne(M, L, C)  
  
    Pour j de 1 à C Faire  
        sommeCol ← 0  
  
        Pour i de 1 à L Faire  
            sommeCol ← sommeCol + (M+i-1)^(j-1) × i  
        FinPour  
  
        T[j] ← partie_entière( sommeCol / moy )  
    FinPour  
Fin
```

- Un sous programme qui applique une conversion de l'image d'origine **M** vers une image avec 3 niveaux de valeurs en effectuant les remplacements suivants :
 - Les valeurs dans (0, ..., 63) sont remplacées par la valeur 0,
 - Les valeurs dans (64, ..., 127) sont remplacées par la valeur 1,
 - Les valeurs dans (128, ..., 191) sont remplacées par la valeur 2,
 - Les valeurs dans (192 ..., 255) sont remplacées par la valeur 3

Procédure Conversion_Image(var M : tableau[L][C] d'entiers, L : entier, C : entier)

Variables

i, j : entier

Pour i ← 1 à L **Faire**

Pour j ← 1 à C **Faire**

Si $0 \leq M[i][j] \leq 63$ **Alors**

$M[i][j] \leftarrow 0$

Sinon Si $64 \leq M[i][j] \leq 127$ **Alors**

$M[i][j] \leftarrow 1$

Sinon Si $128 \leq M[i][j] \leq 191$ **Alors**

$M[i][j] \leftarrow 2$

Sinon

$M[i][j] \leftarrow 3$

FinSi

FinPour

FinPour

FinProcédure

Avec pointeurs

Procédure convertirImage (var M : pointeur sur entier, L : entier, C : entier)

Variables :

i, j : entier

Début

Pour i de 1 à L Faire

Pour j de 1 à C Faire

Si $(M+i-1)^{+j-1} \wedge < 64$ Alors

$(M+i-1)^{+j-1} \leftarrow 0$

SinonSi $(M+i-1)^{+j-1} \wedge < 128$ Alors

$(M+i-1)^{+j-1} \leftarrow 1$

SinonSi $(M+i-1)^{+j-1} \wedge < 192$ Alors

$(M+i-1)^{+j-1} \leftarrow 2$

Sinon

$(M+i-1)^{+j-1} \leftarrow 3$

FinSi

FinPour

FinPour

Fin

Programme Principale

Algorithme Image_Niveaux_Gris

Variables

i,j,L,C : entier

M : tableau[1..100][1..100] d'entiers

T : tableau[1..100] d'entiers

moy : réel

Répéter

Ecrire("Donner le nombre de lignes L (>0) : ")

Lire (L)

Jusqu'à (L > 0)

Répéter

Ecrire ("Donner le nombre de colonnes C (>0) : ")

Lire (C)

Jusqu'à (C > 0)

Saisir_Image(M, L, C)

moy $\leftarrow$ Moyenne_Image(M, L, C)

Generer_Mot_Passe(M, L, C, moy, T)

Conversion_Image(M, L, C)

Ecrire ("Mot de passe T : ")

Pour j $\leftarrow$ 1 à C **Faire**

Ecrire (T[j])

FinPour

Ecrire ("Image convertie :")

Pour i $\leftarrow$ 1 à L **Faire**

Pour j $\leftarrow$ 1 à C **Faire**

Ecrire (M[i][j])

FinPour

FinPour

FinAlgorithme

Programme Principal

Avec pointeurs

Variables :

```
L, C : entier
M : pointeur sur entier
T : pointeur sur entier
moy : réel
j : entier
```

Début

```
Écrire("Donner L : ")
Lire(L)

Écrire("Donner C : ")
Lire(C)

saisirMatrice(M, L, C)

moy ← moyenne(M, L, C)
Écrire("Moyenne des pixels = ", moy)

creerMotDePasse(M, L, C, T)
Écrire("Mot de passe (T) : ")
Pour j de 1 à C Faire
    Écrire( (T + (j-1))^ )
FinPour

convertirImage(M, L, C)
Écrire("Image convertie : ")
```

```
Pour i de 1 à L Faire
    Pour j de 1 à C Faire
        Écrire( (M+i-1)^+j-1)^ )
    FinPour
FinPour

Libérer(M)
Libérer(T)

Fin
```

Exercice 3

- Ecrire un algorithme qui lit un entier X et un tableau A du type int au clavier et élimine toutes les occurrences de X dans A en tassant les éléments restants. Le programme utilisera un pointeur P1.

Exercice 3-correction

Algorithme Supprimer-valeur

Variables :

A : tableau d'entiers [1..100]

N : entier

X : entier à supprimer

P1 : pointeur sur entier

i : entier

taille_finale : entier

Début

// Lecture de la taille du tableau

Lire N

// Lecture du tableau

Pour i $\leftarrow$ 1 à N **faire**

 Lire A[i]

FinPour

// Lecture de l'entier à éliminer

Lire X

// Initialisation du pointeur

P1 $\leftarrow$ &A[1] // pointeur sur le premier élément

taille_finale $\leftarrow$ N

Tant que P1 $\leq$ &A[taille_finale] **faire**

Si *P1 = X **alors**

 // Tasser les éléments suivants

Pour i $\leftarrow$ P1+1 à &A[taille_finale] **faire**

 *(i-1) $\leftarrow$ *i

FinPour

 taille_finale $\leftarrow$ taille_finale - 1

 // Ne pas avancer P1 pour vérifier le nouvel élément à cette position

Sinon

 P1 $\leftarrow$ P1 + 1 // avancer le pointeur

FinSi

FinTantQue

// Affichage du tableau final

Pour i $\leftarrow$ 1 à taille_finale **faire**

 Ecrire(A[i])

FinPour

Fin